LECTURE SCIENTIFIQUE · PLATFORM DOCUMENTATION

Brevo Email Notifications

How the Laravel backend connects to Brevo and delivers every portal notification.

Applies to backend (Laravel) · lecturescientifique.ca **Transport** Brevo SMTP relay **Last verified** 1 August 2026

Email notifications are delivered through Brevo's **SMTP relay** rather than its REST API, which is why there is no Brevo SDK or HTTP client anywhere in the codebase. Laravel's standard mail stack is pointed at Brevo's relay host, and every notification in the platform funnels through a single service class. The sections below follow one message from configuration to delivery.

Delivery chain at a glance

1. Application event (school created, trial ending, password reset…)
2. `NotificationMailer::send*()` builds a Mailable
3. `safeSend()` filters the recipient and calls `Mail::to()->send()`
4. Locale + template lines resolved, markdown view rendered
5. Laravel's SMTP transport connects to `smtp-relay.brevo.com:2525` over STARTTLS
6. Brevo authenticates the SMTP key, DKIM-signs, queues and delivers

1 Brevo side: an SMTP relay account

In the Brevo dashboard, under **SMTP & API**, an SMTP key was generated. This produces two values used as ordinary SMTP credentials:

- A **login** in the form `b22c43001@smtp-brevo.com`
- A **key** beginning with `xsmtpsib-`, which is used as the SMTP password

The sender address `admin@apprendrefrancofun.com` must additionally be registered as a verified sender (or belong to a verified, authenticated domain) in Brevo. Valid credentials alone are not enough — Brevo rejects messages from unverified senders.

2 Credentials live in `.env`

Local and production carry the same block. The values confirmed on the live server are below, with the key masked.

```
MAIL_MAILER=smtp
MAIL_HOST=smtp-relay.brevo.com
MAIL_PORT=2525
MAIL_USERNAME=b22c43001@smtp-brevo.com
MAIL_PASSWORD=<the xsmtpsib- SMTP key>
MAIL_ENCRYPTION=tls
MAIL_FROM_ADDRESS="admin@apprendrefrancofun.com"
MAIL_FROM_NAME="Lecture Scientifique"
```

Port **2525** is Brevo's alternate submission port, chosen because hosting providers frequently block port 25 and sometimes 587. A template of this block ships with the repository at `backend/.env.production.example`, with the password deliberately left blank.

3 Laravel turns the variables into a transport

`config/mail.php` is stock Laravel. `'default' => env('MAIL_MAILER')` selects the `smtp` mailer, and that mailer's block maps each `MAIL_*` variable onto the Symfony Mailer ESMTP transport. The global `from` block is what makes every message originate from the Brevo-verified sender without each Mailable having to declare it.

backend/config/mail.php

```
'smtp' => [
    'transport'  => 'smtp',
    'scheme'     => env('MAIL_SCHEME'),
    'url'        => env('MAIL_URL'),
    'host'       => env('MAIL_HOST', '127.0.0.1'),
    'port'       => env('MAIL_PORT', 2525),
    'username'   => env('MAIL_USERNAME'),
    'password'   => env('MAIL_PASSWORD'),
    'encryption' => env('MAIL_ENCRYPTION', 'tls'),
],
```

OPERATIONAL NOTE

Laravel caches configuration in production, so editing a `MAIL_*` value in `.env` has no effect until the cache is rebuilt. Every deployment therefore ends with `php artisan optimize:clear`.

4 Application code never touches SMTP directly

All notifications funnel through `App\Services\NotificationMailer`. Each event has a small static method — `sendWelcome`, `sendVerifyAccount`, `sendResetPassword`, `sendTrialEnding`, `sendSubscriptionStarted`, `sendSchoolAdded` and others — that builds a Mailable and hands it to one private `safeSend()`. That method does two protective things before Brevo ever sees the message: it skips synthetic portal addresses ending in `@portal.local`, and it wraps the send in a `try/catch` so an SMTP failure is logged rather than breaking the request that triggered it.

backend/app/Services/NotificationMailer.php

```
private static function safeSend(User $user, object $mailable): void
{
    $email = trim((string) $user->email);
    if ($email === '' || str_ends_with(strtolower($email), '@portal.local')) {
        Log::info('Skipped portal email for non-deliverable address', [...]);

        return;
    }

    try {
        Mail::to($email)->send($mailable);
    } catch (Throwable $e) {
        Log::error('Portal email failed', [...]);
    }
}
```

SYNCHRONOUS BY DESIGN

The call is `->send()`, not `->queue()`. The Mailables use the `Queueable` trait, but nothing is queued, so messages are transmitted during the HTTP request — which is why creating a school with three teachers takes a moment (four SMTP conversations inline). `QUEUE_CONNECTION=database` is set, but there is no queue worker or cron on the server, so switching to `->queue()` would silently park mail in the jobs table until a worker is added.

5 Content is assembled per recipient

Each Mailable — `WelcomeEmail`, `TeacherAccountCreatedEmail`, `ResetPasswordEmail`, and the general-purpose `PortalEventEmail` — resolves the recipient's language through `EmailLocale::resolve()` (their `preferred_language`, falling back to the platform default), then pulls every subject and body line through `EmailTemplate::line()`.

That helper checks admin-authored overrides stored in platform settings first, and falls back to `lang/en/emails.php` or `lang/fr/emails.php`. This is the extension point where new templates plug in, such as the `school_added` board notification used by the school creation wizard. The rendered markdown view embeds the platform logo, and Laravel hands the finished MIME message to the transport.

6 The actual connection

Laravel opens a TCP connection to `smtp-relay.brevo.com:2525`, issues `STARTTLS` to upgrade the channel, authenticates with the SMTP login and key, and delivers the message. Verified from the live server:

```
CONNECTION ESTABLISHED
Protocol version: TLSv1.3
Peer certificate: CN = smtp-relay-offshore-us-east1-v2.brevo.com
Verification: OK
250 SIZE 20971520
```

From there Brevo takes over: it applies DKIM signing for the verified sender domain, queues the message on its own infrastructure, and handles delivery and bounce processing. This is what makes the relay preferable to the server's own sendmail — mail leaves from reputable, authenticated IPs and is far less likely to be filtered as spam.

7 Troubleshooting order

STEP	WHERE TO LOOK	WHAT IT TELLS YOU
1. Application	`storage/logs/laravel.log` on the server	A rejected credential or failed handshake appears as `Portal email failed` with the SMTP error text. A filtered recipient appears as `Skipped portal email for non-deliverable address`. If nothing is logged, Laravel handed the message off successfully.
2. Brevo	Transactional → Logs in the Brevo dashboard	Per-message status: accepted, delivered, soft or hard bounced, blocked. Sender-verification problems and daily quota limits surface here.
3. Config cache	`php artisan optimize:clear`	If the API reported emails sent, the log is empty and Brevo shows nothing, the server is almost certainly running on a stale cached config.

8 Reference: notifications sent through Brevo

TEMPLATE KEY	MAILABLE	TRIGGERED BY
`welcome`	WelcomeEmail	New school or board portal account created
`verify`	PortalEventEmail	Portal account activation link
`teacher_created`	TeacherAccountCreatedEmail	Teacher account created by an admin
`reset`	ResetPasswordEmail	Password reset request
`trial_ending` / `trial_expired`	PortalEventEmail	Trial countdown and expiry
`subscription_started`, `_renewed`, `_cancelled`, `_expired`	PortalEventEmail	Subscription lifecycle / Stripe events
`payment_successful` / `payment_failed`	PortalEventEmail	Billing outcomes
`school_added`	PortalEventEmail	New school attached to a board (creation wizard)
`announcement`	PortalEventEmail	Admin system announcement
`inactive_reminder` / `suspension_reminder`	PortalEventEmail	Dormant and at-risk account nudges

Lecture Scientifique — Brevo email notification documentation. Configuration verified against the production server on 1 August 2026. Secrets are intentionally masked; the live SMTP key lives only in `backend/.env` on the server.